

GASSER: Genetic Algorithm for teSt Suite Reduction

Carmen Coviello
University of Basilicata
Potenza, Italy
carmen.coviello@unibas.it

Giuseppe Scanniello
University of Basilicata
Potenza, Italy
giuseppe.scanniello@unibas.it

Simone Romano
University of Bari
Bari, Italy
simone.romano@uniba.it

Giuliano Antoniol
Polytechnique de Montreal
Montreal, Canada
antoniol@ieee.org

ABSTRACT

Background. Regression testing is a practice that ensures a System Under Test (SUT) still works as expected after changes. The simplest regression testing approach is *Retest-all*, which consists of re-executing the entire Test Suite (TS) on the new version of the SUT. When SUT and its TS grow in size, applying Retest-all could be expensive. Test Suite Reduction (TSR) approaches would allow overcoming the above-mentioned issues by reducing TSs while preserving their fault-detection capability.

Aim. In this paper, we introduce **GASSER (Genetic Algorithm for teSt SuiteE Reduction)**, a new approach for TSR based on a multi-objective evolutionary algorithm, namely NSGA-II.

Method. GASSER reduces TSs by maximizing statement coverage and diversity of test cases, and by minimizing the size of the reduced TSs.

Results. The preliminary study shows that GASSER reduces more the TS size with a small effect on fault-detection capability when compared with traditional approaches.

Conclusions. These outcomes highlight the potential benefits of the use of multi-objective evolutionary algorithm in TSR field and pose the basis for future work.

CCS CONCEPTS

• Software and its engineering → Software testing and debugging.

KEYWORDS

Regression Testing, Genetic Algorithm, Test Suite Reduction

ACM Reference Format:

Carmen Coviello, Simone Romano, Giuseppe Scanniello, and Giuliano Antoniol. 2020. GASSER: Genetic Algorithm for teSt Suite Reduction. In *ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '20)*, October 8–9, 2020, Bari, Italy. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3382494.3422157>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEM '20, October 8–9, 2020, Bari, Italy

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7580-1/20/10...\$15.00

<https://doi.org/10.1145/3382494.3422157>

1 INTRODUCTION

Regression testing is a practice that ensures a System Under Test (SUT) still works as expected after changes [25]. The simplest regression testing approach is *Retest-all*, which consists of re-executing, on a changed version of SUT, the entire Test Suite (TS) [17]. As SUT evolves, its TS tends to grow in size and applying Retest-all could require too much time and/or too many resources, making Retest-all a non-practicable option [2]. To overcome these issues, researchers proposed several approaches for regression testing. These approaches can be grouped into three main classes: (i) Test Suite Reduction (TSR); (ii) test case selection; and (iii) test case prioritization [25]. Harrold *et al.* [10] formally defined the TSR problem as follows:

Given: a Test Suite (TS) named T , a set of test requirements $r_1, \dots, r_n$, which must be satisfied to provide the desired testing coverage of the SUT, and subsets of T , $T_1, \dots, T_n$, one associated with each of the r_i s such that any one of the test cases t_j belonging to T_i can be used to test r_i .

TSR Problem: find a representative set, T' , of test cases from T that satisfies all of the r_i s.

Traditional TSR approaches are based on the above-mentioned definition and reduce the size of a TS by satisfying all test requirements. These approaches are named adequate [18]. More recently, researchers have relaxed the constraint of satisfying all test requirements when reducing TSs. Such approaches are named inadequate [18]. That is, rather than satisfying all test requirements, inadequate approaches satisfy a subset of test requirements. Inadequate TSR approaches may lead to a greater reduction in TS size at the expense of a negligible (or even null) loss in fault-detection capability (e.g., [4–6, 18]). This makes inadequate TSR approaches appealing when TSs grow in size. Moreover, most TSR approaches (e.g., [3, 6, 10, 14, 20, 28, 29]) are based on a single dimension—i.e., these approaches use one kind of test requirements (usually code coverage) to reduce TSs. Only a few approaches aim to reduce TSs based on multiple dimensions—i.e., these approaches use more kinds of test requirements (e.g., code coverage and execution cost of test cases). Some empirical investigations have shown that considering multiple dimensions is promising for TSR (e.g., [15, 16, 18, 23, 24]).

In this paper, we present an inadequate approach for TSR, named GASSER (*Genetic Algorithm for teSt SuiteE Reduction*), based on multiple dimensions: (i) statement coverage, (ii) test-case diversity, and (iii) TS size. GASSER identifies reduced TSs that maximize both statement coverage and test-case diversity in the reduced TSs while

minimizing the reduced TSs size. To pursue such a threefold objective, GASSER exploits a multi-objective evolutionary algorithm, namely *NSGA-II* (*Non-dominated Sorting Genetic Algorithm II*) [7].

2 BACKGROUND

2.1 Distance/dissimilarity Measures for Use in the TSR Context

Several distance/dissimilarity measures can be used to compute the diversity of a pair of test cases in terms of the statements they cover when executed. In the following, we summarize the measures that GASSER leverages to compute the test-case diversity and that some researchers have already used in the context of TSR (e.g., [4, 6]).

Euclidean Distance (EUCL). Given two vector $\vec{x} = (x_1, \dots, x_n)$ and $\vec{y} = (y_1, \dots, y_n)$, their Euclidean distance is defined as:

$$d(\vec{x}, \vec{y}) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2} \quad (1)$$

Cosine Dissimilarity (Cos). Let $\vec{x}$ and $\vec{y}$ be two vectors, the cosine dissimilarity between them is defined as:

$$d(\vec{x}, \vec{y}) = 1 - \frac{\vec{x} \cdot \vec{y}}{||\vec{x}|| ||\vec{y}||} \quad (2)$$

Jaccard-based Dissimilarity (JACC). Let **A** and **B** be two sets, the Jaccard-based dissimilarity between them is equal to:

$$d(\mathbf{A}, \mathbf{B}) = 1 - \frac{|\mathbf{A} \cap \mathbf{B}|}{|\mathbf{A} \cup \mathbf{B}|} \quad (3)$$

Hamming Distance (HAMM). Let $\vec{x}$ and $\vec{y}$ be two vectors, their Hamming distance is the number of elements where $\vec{x}$ and $\vec{y}$ differ.

Levenshtein Distance (LEV). Let a and b be two strings (of characters), their Levenshtein distance is the minimum number of operations (i.e., adding a character, deleting a character, and substituting a character with another) that transforms a into b .

K-based Dissimilarity (K). The K-based dissimilarity, namely Cohen's Kappa index, measures the degree of agreement between two raters, who classify items over two or more categories. Let p_o be the observed proportion of agreement and p_e be the agreement expected just by chance, the K-based dissimilarity is defined as:

$$k = 1 - \frac{p_o - p_e}{1 - p_e} \quad (4)$$

String Kernels-based Dissimilarity (SK). Given two strings a and b , the standard formula of the string kernel function is: $k(a, b) = \sum_{s \in A^+} num_s(a) num_s(b) \lambda_s$, where A^+ represents the set of non-empty substrings; λ_s is a chosen weight or decay factor; and $num_s(a)$ and $num_s(b)$ are the number of occurrences of the substring s in a and b . In our work, we used the *bounded-range kernel* function, where $\lambda_s = 0$ for all $|s| > n$ and n is fixed [13]. This means that only matching substrings of length less than or equal to n are considered. The string kernels-based dissimilarity between a and b is computed as:

$$d(a, b) = 1 - k^*(a, b) \quad (5)$$

where $k^*(a, b)$ is the normalized (bounded-range) string kernel function that returns values in between 0 and 1.

To compute the diversity of a pair of test cases in terms of statement coverage, it is required to define an appropriate encoding for the statements a given test case covers. For the Euclidean distance, cosine dissimilarity, and Hamming distance, test case is encoded as

a binary vector $\vec{x}$. The dimension of this vector is equal to the number of statements in the SUT. When a test case covers a statement, the corresponding element in the binary vector is equal to one, zero otherwise. For the Jaccard-based dissimilarity, we represent the test case covered statements as a set. The encoding for K-based dissimilarity is also based on a binary vector but an additional step is needed to compute p_e and p_o (see Equation 4). In particular, for each pair of test cases t_1 and t_2 , we concatenate their corresponding binary vectors, $\vec{x}_1$ and $\vec{x}_2$, to build the binary vector $\vec{x}_{1,2}$. Then, we computed p_{one} and p_{zero} (i.e., the total number of ones and zeros in $\vec{x}_{1,2}$ divided by the length of $\vec{x}_{1,2}$, respectively) to obtain p_e , which is defined as follows: $p_e = p_{one}^2 + p_{zero}^2$. Whereas, p_o is computed as the number of statements that both t_1 and t_2 cover plus the number of statements that both t_1 and t_2 do not cover divided by the length of $\vec{x}_{1,2}$. For Levenshtein distance and string kernels-based dissimilarity, the test case covered statements are encoded as a string, where each character represents a given covered statement.

2.2 Genetic Algorithms

Search-Based Optimization (SBO) techniques are usually preferred over traditional optimization ones (e.g., linear programming and dynamic programming) when the set of possible solutions (i.e., search space) is large, no close mathematical problem formulation exists, or computing the exact solution is too expensive [9]. This implies that SBO techniques find near-optimal solutions usually close to the optimal ones although they do not guarantee the convergence to the global optimum. A Genetic Algorithm (GA) is a SBO technique based on Darwin's Principle of Natural Selection. GAs select the currently better solutions and recombine them to create new solutions nearer to the optimal one. A GA iteratively evolves an initial population, which is a set of candidate solutions of the considered problem, for searching over the solution space of the problem. The population evolves by applying genetic operators (e.g., crossover and mutation). To determine the quality of the solutions, a fitness function(s) has to be defined. In the population evolution, solutions with a higher fitness values are more likely to be preserved and evolved. In other words, the fittest individuals of the population are selected to pass their genes to the next generation. The selected individuals are paired (and called parents) and for each pair, a crossover point is randomly chosen within the genes. Some offspring are created swapping the parents' genes until they reach the crossover point. These offspring are added to the population. The population undergoes a mutation phase, where some of the offspring genes can be mutated with a low random probability. Mutation operation occurs to maintain diversity within the population and prevent premature convergence. The iterations of a GA terminate when either a maximum number of populations have been produced or a satisfactory fitness level has been reached for the population. During the years GAs have been largely used since they are broadly applicable and easily tailored to specific problems like regression testing and TSR [15, 16, 24, 25].

3 RELATED WORK

Traditional adequate TSR approaches are based on a single dimension and use heuristics to reduce TSs. Harrold *et al.* [10] proposed *Harrold-Gupta-Soffa* (HGS), a heuristic-based approach for TSR. At each iteration i , HGS considers the set of test requirements that only

i test cases satisfy, thus it selects these test cases. Li *et al.* [14] proposed *Greedy (GRD)*, an approach based on a greedy algorithm to prioritize test cases which later was applied to reduce TSs [19]. GRD selects each time the test case, among those available, that satisfies the highest number of test requirements that are yet unsatisfied. Li *et al.* [14] also introduced *2-Optimal (2OPT)*, based on the 2-Optimal algorithm. Tallam and Gupta [20] defined *Delayed Greedy (DGR)*, a variant of GRD that perform additional operations before applying the greedy heuristic. Chen and Lau [3] proposed *GE* and *GRE*, two heuristic-based approaches both representing a variant of GRD. These approaches rely on the concept of *essential test case* to reduce TSs, namely: if a test requirement r_i can be satisfied by only test case t_i , then t_i is an essential test case. Coviello *et al.* [5] proposed a *Clustering-Based (CB)* approach for TSR and empirically assessed their proposal. The CB approach was instantiated by using several distance/dissimilarity measures. Later, the authors [6] extended the empirical assessment by considering seven instances of CB approach and six traditional TSR approaches (*i.e.*, HGS, GRD, DGR, 2OPT, GE, and GRE) in their adequate and inadequate variants—each studied approach considered a single dimension. The results suggested that a subset of studied TSR approaches behave similarly and outperform the other approaches in terms of trade-off between the reduction in TS size and loss in fault-detection capability. This subset comprises four inadequate variants of traditional approaches (*i.e.*, 2OPT, GE, GRE, and GRD) and five inadequate instances of CB approach (*i.e.*, CB_{COS} , CB_{EUC} , CB_{JACC} , CB_{HAMM} , and CB_K). Yoo and Harman [24] showed that approaches based on greedy algorithms are not always Pareto efficient when optimizing multiple objectives as compared to GA-based approaches, and suggested the use of GA in the context of both test case selection and TSR. Shi *et al.* [18] studied traditional adequate TSR approaches and their inadequate variants to verify the benefits that can be achieved when relaxing the constraint of covering all test requirements. The authors also investigated the use of both a single dimension (*i.e.*, statement or mutant coverage) and two dimensions (*i.e.*, statement and mutant coverage) for TSR. Marchetto *et al.* [15] proposed *MORE+* an approach that considers three dimensions for TSR: code coverage; functional-requirement coverage; and execution cost of test cases. The results of an empirical assessment indicated that the reduced TSs identified by applying MORE+ detected more faults as compared to the baseline approaches the authors chose. The main drawback of multi-dimension approaches is that they require more sources of information (*e.g.*, both statement and mutant coverage) to perform TSR as compared with GASSER that only exploits a single source of information (*i.e.*, statement coverage). Thus, GASSER can be more easily applicable in an actual regression testing scenarios.

4 THE GASSER APPROACH

GASSER exploits a multi-objective GA, a multi-objective evolutionary algorithm named NSGA-II [7]. It uses a set of genetic operators (*i.e.*, selection, crossover, and mutation) to iteratively evolve an initial population of candidate solutions (*i.e.*, each candidate solution is a reduced TS). The evolution is guided by objective functions (also called fitness functions) that evaluate each candidate solution with respect to chosen dimensions. At each iteration, the Pareto front of the best alternative solutions is generated from the evolved population. The Pareto front contains the set of non-dominated

solutions. A solution T'_1 is said to dominate another solution T'_2 if both the following conditions are verified: (i) T'_1 is no worse than T'_2 for all dimensions; and (ii) T'_1 is strictly better than T'_2 for at least one dimension. Population evolution is iterated until either maximum number of iterations is reached or convergence is reached (*i.e.*, when a satisfying fitness level is reached for the population). GASSER considers three dimensions (*i.e.*, statement coverage, test-case diversity, and TS size) to reduce TSs. In particular, GASSER aims to: (i) maximize the number of statements covered by the reduced TSs; (ii) maximize the diversity of test cases in the reduced TSs; and (iii) minimize the number of test cases in the reduced TSs.

If we consider GASSER as a black-box, it takes as input a statement coverage matrix where rows and columns represent, respectively, statements and test cases of the SUT. An entry (i, j) of the matrix indicates if the statement i is covered by the test case j , namely the value of the entry (i, j) is one if test case j covers the statement i , zero otherwise. In the TSR field, without loosing in generality, it is customary that the information on the covered statement is available from the last tested version of a SUT (*e.g.*, [15]). The output is a set of reduced TSs, namely all the non-dominated solutions of the Pareto front [24].

To pursue the threefold goal of maximizing both statement coverage and test-case diversity, and minimizing TS size, GASSER instantiates NSGA-II as follows.

Solution Encoding. A reduced TS is encoded as a binary string. Each character represents a test case of the TS and is set to one if the test case is included in the TS, zero otherwise.

Initialization. The starting population is defined by randomly selecting reduced TSs among all those possible.

Genetic Operators. GASSER uses the binary-tournament selection operator, namely: given two solutions (*i.e.*, two reduced TSs), the solution with higher ranking is selected¹ (the ranking is given by the fitness functions). After applying the selection operator, the crossover and mutation operators are applied to allows the population to evolve. In particular, we used the one-point crossover in which the crossover point is randomly selected for a pair of parents. The crossover point (or cut-point) is the point from which the two tails of parents are then recombined to generate two new solutions (*i.e.*, the offspring). To guarantee variability in the population and to prevent premature convergence, GASSER applies a bit-flip mutation operator to certain formed offspring so that some elements of the offspring are subjected to a mutation with a low random probability. In particular, the bit-flip mutation operator randomly flips (*e.g.*, one becomes zero and vise-versa) some elements of the offspring.

Objective Functions. Given the threefold goal of GASSER, the candidate solutions (*i.e.*, the reduced TSs) are evaluated by using three fitness functions: C (Coverage), D (Diversity), and S (Size). C is defined as:

$$C(T'_i) = \sum_{j=1}^{|T'_i|} Cov(t_j) \quad (6)$$

where T'_i is the i -th reduced TS, and Cov is a function that computes the covered statements of the test case t_j . Therefore, C is the cumulative number of statements that test cases of reduced TS cover. The higher the value, the better it is since the goal is to maximize the covered statement by a reduced TS.

¹Note that, if the ranking is the same, a solution is selected if its crowding distance is greater than the other.

D is computed as the sum of the diversity of all the possible pairs of test cases of a reduced TS:

$$D(T'_i) = \sum_{i=j}^{|T'_i|} \sum_{k=j+1}^{|T'_i|} Div(t_j, t_k) \quad (7)$$

where Div is a function that computes the diversity between a pair of test cases by applying one of the measures presented in Section 2.1 (e.g., Euclidean Distance). The higher the value, the better it is as our goal is to maximize the test case in reduced TS.

Finally, S is the test cases number of a reduced TS: $S(T'_i) = |T'_i|$. The lower the value, the better it is since the goal is to minimize reduced TS size.

5 EXPERIMENT

We planned and executed our experiment by following Wohlin *et al.*'s guidelines [21].

5.1 Definition and Context

We defined the goal of our experiment, by using the GQM (Goal-Question-Metric) template [1], as follows:

Analyze GASSER **for the purpose of** comparing it with state-of-the-art TSR approaches **with respect to** reduction in TS size and loss in fault-detection capability of the reduced TSs **from the point of view of** the researcher and of the practitioner **in the context of** open-source SUT implemented in Java.

Given this goal, we formulated and studied the following Research Question (RQ):

RQ. As compared to state-of-the-art TSR approaches (i.e., the baseline approaches for comparison), does GASSER allow achieving a better trade-off in terms of reduction in TS size and loss in fault-detection capability?

We selected 19 versions of four Java software systems (i.e., the considered experimental objects) from SIR,² a public repository that supports experiments in the regression testing field [8]. SIR has been widely used for empirical assessment of regression testing approaches, including TSR approaches (e.g., [4, 6, 16, 28]). SIR provides, for any experimental object, the faults the original TSs reveal so allowing computing the reduction in fault-detection capability of the studied TSR approaches. Among the systems in the SIR dataset, we focused on those implemented in Java provided with JUnit TSs. The experimental objects are: eight versions of *Ant*, three versions of *jTopas*, five versions of *jMeter*, and three versions of *XMLSecurity*. These experimental objects are the same as past empirical studies conducted in the TSR field (e.g., [4, 6, 28]).

5.2 Planning

Coviello *et al.* [6] observed that, among the TSR approaches they studied, there is a subset of approaches that behave similarly and outperform the other approaches in terms of trade-off between reduction in TS size and loss in fault-detection capability (Section 3). Based on these results, we considered this subset of TSR approaches as a baseline for comparison. Thus, we studied four inadequate variants of traditional approaches (i.e., 2OPT [14], GE [3], GRD [14], and GRE [3]) and five inadequate instances of the CB approach by Coviello *et al.* [6] (i.e., CB_{COS}, CB_{EUCL}, CB_{HAMM}, CB_{JACC}, and

CB_K). Coviello *et al.* [6] made available their experimental data on the Web and used the same experimental objects as we used in our experiment. Therefore, we exploited their data as a kind of benchmark to compare our approach and the baselines, without implementing and running them on the experimental objects.

As for the GASSER approach, we instantiated GASSER by using each time a different distance/dissimilarity measure, i.e., those shown in Section 2.1. Therefore, the considered instances of GASSER are: GASSER_{COS}, GASSER_{EUCL}, GASSER_{HAMM}, GASSER_{JACC}, GASSER_K, GASSER_{LEV}, GASSER_{SK}. For example, GASSER_{EUCL} indicates the instance of the GASSER approach that relies on the Euclidean Distance to compute test-case diversity. Given an experimental object, the output of each instance of GASSER is a set of reduced TSs. From the set of non-dominated solutions, we need to choose a single solution to compare the GASSER instances with the baseline approaches. To this end, we adopted a criterion based on the compromise programming method [26, 27]. The used criterion requires to plot each non-dominated solution in a three-dimensional space where the x , y , and z axes represent statement coverage, test-case diversity, and TS size, respectively. Let max_x and max_y be the maximums for the x and y axes (dimensions to be maximized), we computed the Euclidean Distance between each non-dominated solution and the point $(max_x, max_y, 0)$, namely the ideal point that is, in general, not attainable [26, 27]. The non-dominated solution closest to the ideal point is the solution that achieves the best trade-off among the considered dimensions.

5.3 Variables and Data Analyses

In our experiment, the main independent variable is **Approach**. This variable is nominal and indicates the TSR approaches we investigated. The Approach variable assumes the following values: 2OPT, CB_{COS}, CB_{EUCL}, CB_{HAMM}, CB_{JACC}, CB_K, GASSER_{COS}, GASSER_{EUCL}, GASSER_{HAMM}, GASSER_{JACC}, GASSER_K, GASSER_{LEV}, GASSER_{SK}, GE, GRD, or GRE.

Past work has mainly assessed TSR approaches based on two metrics (e.g., [4, 11, 17, 25, 28]), namely: Reduction in test suite Size (**RS**) and Reduction in Fault-detection capability (**RF**). Let $|T|$ and $|T'|$ be the size of original and reduced TSs (i.e., T and T'), respectively. Let F and F' be the number of faults original and reduced TSs detect, respectively. RS and RF are defined as follows:

$$RS = \frac{|T| - |T'|}{|T|} \times 100 \quad RF = \frac{F - F'}{F} \times 100 \quad (8)$$

RS assumes values in $[0, 100]$ and indicates the percentage of test cases in the original TS that are not included in the reduced one. The higher the RS value, the greater the reduction in TS size is. RF ranges in $[0, 100]$ and indicates the percentage of faults that reduced TS does not detect if compared to the original TS. The higher the RF value, the greater the loss in fault-detection capability is. The goal of a given TSR is to preserve the fault-detection capability of the original TS while reducing its size [10]. Accordingly, we can estimate the trade-off for a TSR approach in terms of reduction in TS size and loss in fault-detection capability the TSR approach achieves by assessing the RS and RF values together. The best-case scenario is when the RF values are close to zero with high RS values. A TSR approach A achieves a trade-off between reduction in TS size and loss in fault-detection capability better than an approach B when

² sir.unl.edu

RF assumes similar values for both approaches A and B and the RS value for A is better (*i.e.*, higher) than the RS value for B . We also consider A better than B when B achieves a substantially worse (*i.e.*, higher) RF value whatever the RS values for A and B are.

To answer our RQ, we computed descriptive statistics—*i.e.*, median, mean, and Standard Deviation (SD)—summarizing the distributions of the RS and RF values of the studied TSR approaches. We also used boxplots to graphically summarized these distributions.

5.4 Instrumentation

As for the RS and RF values for the baseline approaches, we downloaded the experimental data Coviello *et al.* [6] made available on the Web. As for GASSER, we implemented a prototype of a supporting system. It integrates the NSGA-II implementation available in jMetal.³ We configured that implementation as follows: *max population size* = 100; and *max number of iterations* = 1000. Since NSGA-II has a non-deterministic behavior, we ran GASSER 10 times on each experimental object and collected the data regarding the reduced TSs. The statement coverage information of the experimental objects GASSER takes as input was gathered by exploiting JaCoCo⁴ as Coviello *et al.* did.

5.5 Threats to Validity

Internal validity threats concern factors internal to the experiment [21]. The implementation of both GASSER and the baseline approaches might threaten the validity of the results. To mitigate this kind of threat, we tested our prototype of a supporting system implementing GASSER before applying it. As for the baseline approaches, we do not have any control on their implementations. The procedures used to collect the data and compute the values of the metrics might also threaten the validity of the results.

Construct validity threats concern the relationship between theory and observation [21]. A possible threat to construct validity is the use of a single measure to assess a construct. However, RS and RF represent the standard to quantify reduction in TS size and loss in fault-detection capability, respectively (*e.g.*, [11, 17, 22]).

Conclusion validity is concerned with issues affecting the ability to draw correct conclusions about relations between treatment and experiment outcome [21]. The reliability of the used measures might also affect conclusion validity. As mentioned before, these measures represent the standard for the assessment for TSR approaches. Another threat to conclusion validity concerns the data of SIR dataset. However, SIR was created by external researchers [8] and used in several studies (*e.g.*, [12, 16, 28]). The use of the criterion to select one non-dominated solution among those the GASSER instances identified represents another threat to conclusion validity.

External validity threats concern the possibility of generalizing results [21]. Although in our experiment we studied software systems from SIR that cover different application domains, we cannot be sure that our findings can be generalized to the universe of Java systems. We plan to conduct future work to verify to what extent our findings hold for other kinds of experimental objects (*e.g.*, industrial software systems). The approaches chosen as a baseline for comparison might threaten external validity too. These approaches

Table 1: Descriptive statistics for RS and RF.

Approach	RS (%)			RF (%)		
	Median	Mean	SD	Median	Mean	SD
ZOPT	25.27	29.66	13.25	0	0	0
CB _{Cos}	28.39	30.6	7	0	0.66	2.87
CB _{Eucl}	29.5	27.99	8.12	0	0.75	3.28
CB _{Hamm}	28.71	29.34	7.48	0	0.75	3.28
CB _{Jacc}	28.62	29.26	8.08	0	0.66	2.87
CB _K	27.08	27.12	5.93	0	0.75	3.28
GASSER _{Cos}	35.71	34.2	10.2	0	7.58	14.66
GASSER _{Eucl}	40.09	38.77	7.05	0	16.11	26.5
GASSER _{Hamm}	41.47	40.59	5.3	0	4.95	10.53
GASSER _{Jacc}	38.1	38.35	5.48	0	8.33	16.09
GASSER _K	40.57	38.35	7.33	0	8	17.89
GASSER _{LEV}	26.48	27.48	6.55	0	0.75	3.28
GASSER _{SK}	40.35	39.81	6.52	0	15.78	25.71
GE	26.37	29.17	13.82	0	0	0
GRD	26.37	29.57	13.52	0	0.88	2.71
GRE	24.47	28.98	13.94	0	0.87	2.67

represent a natural choice since they have been used in past studies and are considered the state-of-the-art (*e.g.*, [6, 18, 19, 28]). Also, the selection of the baseline approaches was based on the results of a past study [6].

6 RESULTS AND DISCUSSION

In Table 1, we report descriptive statistics for each approach and each dependent variable. As for RS, we can observe that GASSER instances allow obtaining higher median and mean values as compared to the baseline approaches, except for GASSER_{LEV}. Indeed, GASSER_{LEV} has mean and median values comparable with or worse than the baseline approaches although its SD value is better. Boxplots in Figure 1 (left-hand side) confirm this finding and suggests that GASSER_{Hamm} is the best configuration in terms of reduction in TS size—the box is less skewed and shorter than the others and its median value is better. As for RF, we can observe that the baseline approaches allow obtaining better mean values for RF than the GASSER instances (see Table 1) with the only exception of GASSER_{LEV}. The only GASSER instances that produced RF values not so far from the baselines (inferior to 10% on average) are: GASSER_{Cos}, GASSER_{Hamm}, GASSER_{Jacc}, and GASSER_K. For each approach, the median values are equal to zero. The boxplots in Figure 1 (right-hand side) support these findings. Indeed, the boxplots indicate that the best GASSER instance is GASSER_{LEV} followed by GASSER_{Hamm} and GASSER_K.

Summarizing, descriptive statistics and boxplots suggest that the best approach in terms of trade-off between RS and RF is GASSER_{Hamm}. It allowed obtaining the highest reduction in TS size (RS equal to 40.59% on average) with a small effect on loss in fault-detection capability (RF equal to 4.95% on average). As compared to GASSER_{Hamm}, the baseline approaches allow obtaining lower RF values but at the cost of lower RS values.

Remarks. We expect that a TSR approach seeks to preserve fault-detection capability of the original TS while obtaining a remarkable reduction of TS size. The preliminary results of our experiment suggest that GASSER_{Hamm}, GASSER_{Jacc}, and GASSER_K are the instances that allow obtaining better trade-off in terms of RS and RF. That is, we can positively answer RQ: GASSER_{Hamm}, GASSER_{Jacc}, and GASSER_K allow obtaining better trade-off in terms of RS and RF as compared to both other GASSER instances and baseline approaches. Our findings are even stronger if we consider that we compared the GASSER instances with the best TSR approaches identified by Coviello *et al.* [6]. Finally, preliminary results justify

³jmetal.sourceforge.net

⁴jacoco.org

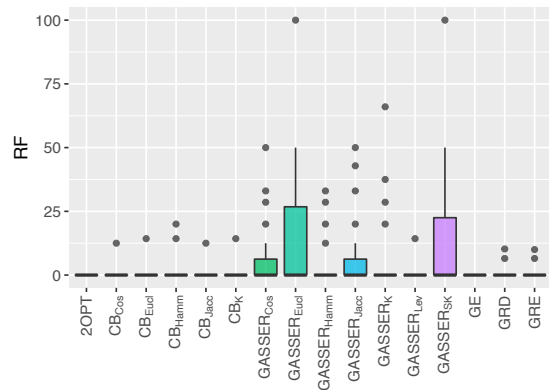
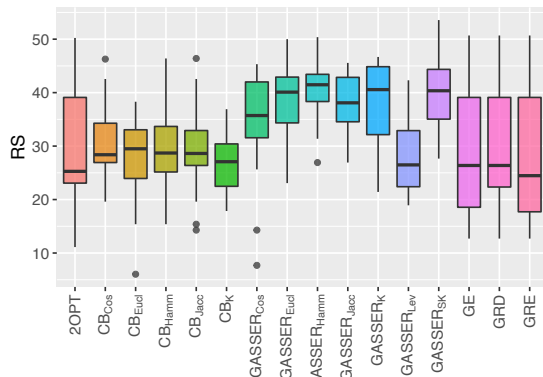


Figure 1: Boxplots of the RS and RF values for each TSR approach.

our research on the use of a multi-objective evolutionary algorithm, namely NSGA-II to reduce TSs by maximizing statement coverage, diversity of test cases, and minimizing reduced TSs size. In the future, we plan to extend preliminary empirical assessment to deal with conclusion (e.g., extending the inferential statistic analysis) and external validity (e.g., considering additional experimental objects) threats. The goal of delineated future work is to improve the body of knowledge on the use of evolutionary and multi-objective algorithms applied to TSR. Preliminary investigation poses the basis and justifies planned directions for future work.

7 CONCLUSIONS

We presented a new approach for TSR, GASSER (Genetic Algorithm for teSt SuiteE Reduction), which reduces TSs by considering: statement coverage, test-case diversity, and TS size. Indeed, we presented seven instances of GASSER. Each of them uses a different measure to assess test-case diversity. To preliminarily assess the validity of GASSER, we conducted an experiment in which we compared GASSER with nine baseline approaches in terms of reduction in TS size and loss in fault-detection capability of the reduced TSs. The most important result is: some instances of GASSER, compared with the baselines, reduce more the TSs size with a small effect on the capability to reveal faults.

REFERENCES

- [1] V. Basili, G. Caldiera, and D. H. Rombach. 1994. *The Goal Question Metric Paradigm*, *Encyclopedia of Software Engineering*. John Wiley and Sons. 528–532 pages.
- [2] Swarnendu Biswas, Rajib Mall, Manoranjan Satpathy, and Srihari Sukumaran. 2011. Regression Test Selection Techniques: A Survey. *Informatica* 35, 3 (2011), 289–321.
- [3] Tsong Yueh Chen and Man Fai Lau. 1996. Dividing Strategies for the Optimization of a Test Suite. *Inf. Process. Lett.* 60, 3 (Nov. 1996), 135–141.
- [4] Carmen Coviello, Simone Romano, and Giuseppe Scanniello. 2018. An Empirical Study of Inadequate and Adequate Test Suite Reduction Approaches. In *Proc. of International Conference on Empirical Software Engineering and Measurement*. ACM, 12:1–12:10.
- [5] Carmen Coviello, Simone Romano, Giuseppe Scanniello, Alessandro Marchetto, Giuliano Antoniol, and Anna Corazza. 2018. Clustering Support for Inadequate Test Suite Reduction. In *Proc. of International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 95–105.
- [6] Carmen Coviello, Simone Romano, Giuseppe Scanniello, Alessandro Marchetto, Anna Corazza, and Giuliano Antoniol. 2020. Adequate vs. inadequate test suite reduction approaches. *Inf. Softw. & Technol.* 119 (2020).
- [7] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* 6, 2 (2002), 182–197.
- [8] Hyunsook Do, Sebastian G. Elbaum, and Gregg Rothermel. 2005. Supporting Controlled Experimentation with Testing Techniques: An Infrastructure and its Potential Impact. *Empirical Softw. Engg.* 10, 4 (2005), 405–435.
- [9] Mark Harman and John Clark. 2004. Metrics Are Fitness Functions Too. In *Proc. of International Symposium on Software Metrics*. IEEE, 58–69.
- [10] M. Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. 1993. A Methodology for Controlling the Size of a Test Suite. *ACM Trans. Softw. Eng. Methodol.* 2, 3 (1993), 270–285.
- [11] D. Jeffrey and Neelam Gupta. 2005. Test suite reduction with selective redundancy. In *Proc. of International Conference on Software Maintenance*. IEEE, 549–558.
- [12] B. Jiang, Z. Zhang, T. H. Tse, and T. Y. Chen. 2009. How Well Do Test Case Prioritization Techniques Support Statistical Fault Localization. In *Proc. of International Computer Software and Applications Conference*. IEEE, 99–106.
- [13] Alexandros Karatzoglou and Ingo Feinerer. 2010. Kernel-based machine learning for fast text mining in R. *Computational Statistics & Data Analysis* 54, 2 (2010), 290–297.
- [14] Zheng Li, Mark Harman, and Robert M. Hierons. 2007. Search Algorithms for Regression Test Case Prioritization. *IEEE Trans. Softw. Eng.* 33, 4 (2007), 225–237.
- [15] A. Marchetto, G. Scanniello, and A. Susi. 2019. Combining Code and Requirements Coverage with Execution Cost for Test Suite Reduction. *IEEE Trans. Software Eng.* 45, 4 (2019), 363–390.
- [16] Annibale Panichella, Rocco Oliveto, Massimiliano Di Penta, and Andrea De Lucia. 2015. Improving Multi-Objective Test Case Selection by Injecting Diversity in Genetic Algorithms. *IEEE Trans. Software Eng.* 41, 4 (2015), 358–383.
- [17] Gregg Rothermel, Mary Jean Harrold, Jeffery von Ronne, and Christie Hong. 2002. Empirical studies of test-suite reduction. *Softw. Test. Verif. Reliab.* 12, 4 (2002), 219–249.
- [18] August Shi, Alex Gyori, Milos Gligoric, Andrey Zaytsev, and Darko Marinov. 2014. Balancing Trade-offs in Test-suite Reduction. In *Proc. of International Symposium on Foundations of Software Engineering*. ACM, 246–256.
- [19] Adam M. Smith and Gregory M. Kapfhammer. 2009. An Empirical Study of Incorporating Cost into Test Suite Reduction and Prioritization. In *Proc. of Symposium on Applied Computing*. ACM, 461–467.
- [20] Sriraman Tallam and Neelam Gupta. 2005. A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization. In *Proc. of Program Analysis for Software Tools and Engineering*. ACM, 35–42.
- [21] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, and A. Wesslén. 2012. *Experimentation in Software Engineering*. Springer.
- [22] W. Eric Wong, Joseph R. Horgan, Saul London, and Aditya P. Mathur. 1995. Effect of Test Set Minimization on Fault Detection Effectiveness. In *Proc. of International Conference on Software Engineering*. ACM, 41–50.
- [23] Xue-ying MA, Zhen-feng He, Bin-kui Sheng, and Cheng-qing Ye. 2005. A genetic algorithm for test-suite reduction. In *Proc. of International Conference on Systems Man and Cybernetics*, Vol. 1. 133–139.
- [24] Shin Yoo and Mark Harman. 2007. Pareto efficient multi-objective test case selection. In *Proc. of International Symposium on Software testing and Analysis*. ACM, 140–150.
- [25] S. Yoo and M. Harman. 2012. Regression Testing Minimization, Selection and Prioritization: A Survey. *Softw. Test. Verif. Reliab.* 22, 2 (2012), 67–120.
- [26] P. L. Yu. 1973. A Class of Solutions for Group Decision Problems. *Management Science* 19, 8 (1973), 936–946.
- [27] Milan Zelany. 1974. A concept of compromise solutions and the method of the displaced ideal. *Computers & Operations Research* 1, 3 (1974), 479–496.
- [28] Lingming Zhang, Darko Marinov, Lu Zhang, and Sarfraz Khurshid. 2011. An Empirical Study of JUnit Test-Suite Reduction. In *Proc. of International Symposium on Software Reliability Engineering*. IEEE, 170–179.
- [29] Hao Zhong, Lu Zhang, and Hong Mei. 2006. An experimental comparison of four test suite reduction techniques. In *Proc. of International Conference on Requirements Engineering*. ACM, 636–640.